LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

CVE-2026-49762 Unbounded integer parsing in the Version module enables CPU and memory exhaustion denial of service

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-15
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

Published: 2026-06-15 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

CVE-2026-49762 Unbounded integer parsing in the Version module enables CPU and memory exhaustion denial of service

1. Executive Summary

CVE-2026-49762 is a medium-severity denial-of-service flaw in Elixir's standard-library `Version` parser. The public CNA, GitHub advisory, OSV, and NVD enrichment agree that numeric version components are converted to arbitrary-precision integers without a length bound. When an application passes attacker-controlled version strings into functions such as `Version.parse/1`, `Version.match?/3`, or `Version.parse_requirement/1`, a single oversized all-digit component can pin a BEAM scheduler in non-yielding integer conversion work and, at larger sizes, raise an uncaught `SystemLimitError` that crashes the calling process.

This is not a classic Microsoft-authored vulnerability even though it entered the current workflow through the Microsoft Security Response Center (MSRC) update guide. MSRC's June 2026 CVRF feed is tracking the issue because Azure Linux 3.0 shipped an affected `elixir` package (`1.16.1-1`) and published a vendor fix in `1.16.1-2`. The underlying bug is upstream Elixir logic that affects versions from `1.5.0` before `1.20.1`, so defenders should assess both Microsoft-packaged Linux estates and any self-managed Elixir/Phoenix applications that parse untrusted semantic-version input.

The strongest immediate actions are to patch to Elixir `1.20.1` or vendor backports carrying the same fix, identify applications that accept attacker-influenced version strings through HTTP parameters, manifests, package metadata, or API payloads, and add telemetry for oversized semver-like input and `SystemLimitError`/`Version.Parser` failures. Public sources reviewed during this task did not provide authoritative evidence of CISA KEV inclusion or broad in-the-wild exploitation, but the flaw is operationally relevant because a single request or record can be enough to exhaust CPU and memory in exposed application paths.

2. Intelligence Requirement

This report addresses the following intelligence requirement: what does CVE-2026-49762 mean for defenders running Elixir directly or through downstream packages such as Azure Linux and Debian, which workflows are materially exposed, and what remediation, detection, and hunting actions should be prioritized now?

Question	Assessment
What is the flaw?	Unbounded parsing of numeric version components inside Elixir's `Version` module causes excessive arbitrary-precision integer work and potential process termination through `SystemLimitError`.
What is the primary impact?	CPU and memory exhaustion leading to application slowdown, scheduler pinning, or process crash.
Which public entry points are affected?	`Version.parse/1`, `Version.parse!/1`, `Version.match?/3`, `Version.compare/2`, and `Version.parse_requirement/1`.
Which upstream versions are affected?	Elixir `>= 1.5.0` and `< 1.20.1` per the GitHub security advisory and CVE JSON 5 CNA record.
What is the Microsoft-specific exposure?	MSRC lists Azure Linux 3.0 `elixir` `1.16.1-1` as known affected and `1.16.1-2` as the vendor-fixed package.
Is exploitation necessarily remote?	No. The CNA rates the flaw `AV:L`, but many real applications pass network-supplied strings into the vulnerable parser, making remote triggering possible in exposed business logic.
Is broad exploitation confirmed?	No authoritative public evidence of KEV inclusion or broad in-the-

	wild exploitation was identified during this review.
What should defenders do next?	Patch, inventory exposed parsing paths, constrain user-controlled version parsing, and hunt for oversized semver-like input plus parser-related failures.

3. Source Review and Confidence

Source	Contribution	Confidence
MSRC update guide + June 2026 CVRF	Confirms Microsoft tracking context, Azure Linux affected package, and Microsoft vendor-fix package version	High
GitHub security advisory `GHSA-w2h8-8x3g-278p`	Primary upstream advisory for affected versions, impact language, and patched release boundary	High
CVE JSON 5 / CNA record	Canonical CNA description, affected version range, CWE, references, and CVSS v4.0 data	High
Elixir `v1.20.1` release notes	Confirms shipped remediation language: numeric `Version` components are limited to 14 digits	High
Upstream patch commit `c64417d72fd5c7d09e963ca3ac5fa2b140978d9e`	Direct engineering evidence of the parser limit and associated regression tests	High
NVD entry / JSON 2.0 API	Adds enriched technical description and current public status metadata (`Deferred`)	Medium-High
Debian Security Tracker	Corroborates downstream package lag and release-specific distro status	High
OSV record `EEF-CVE-2026-49762`	Confirms aliasing between CVE and GHSA plus upstream fixed commit range	Medium-High

Confidence is high on flaw mechanics, affected version boundaries, and remediation because the reviewed material includes the Elixir advisory, CNA record, OSV, upstream patch, and Microsoft CVRF packaging data. Confidence is lower only on exploitation prevalence: the reviewed public record is rich in vulnerability mechanics but thin on incident reporting, named campaigns, or broad field exploitation data.

One analytic caveat matters for defenders reading the Microsoft feed. MSRC is functioning here as a downstream package tracker, not as the original vulnerability authority. That means Azure Linux package state is important for Microsoft-aligned estates, but the broader remediation boundary still comes from upstream Elixir and any vendor backports that carry the same parser-limit fix.

4.1 Vulnerability metadata

Field	Value
CVE	`CVE-2026-49762`
Product family	Elixir
Affected component	Standard-library `Version` / `Version.Parser` logic in

	`lib/version.ex`
Weakness	`CWE-400 Uncontrolled Resource Consumption`
Public severity	`CVSS v4.0 5.1 / MEDIUM`
CVSS vector	`CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`
Upstream affected versions	`>= 1.5.0` and `< 1.20.1`
Upstream patched version	`1.20.1`
Microsoft-affected package	Azure Linux 3.0 `elixir` `1.16.1-1`
Microsoft vendor fix	Azure Linux 3.0 `elixir` `1.16.1-2`
Debian status snapshot	`bullseye`, `bookworm`, `trixie`, and `forky` listed vulnerable; `sid` listed fixed at `1.20.1.dfsg-2`
Published date	2026-06-09 (CNA/GHSA publication window; MSRC package visibility updated 2026-06-15)
Exploitation status	No reviewed evidence of KEV inclusion or broad in-the-wild exploitation

4.2 Flaw mechanics

The vulnerable logic accepts numeric version components of unbounded length and converts them to integers. Public CNA and NVD text state that one large all-digit component can force super-linear, non-yielding `String.to_integer/1` / `:erlang.binary_to_integer/1` work that monopolizes a BEAM scheduler, while a larger component can raise an uncaught `SystemLimitError` and crash the calling process.

The issue is reachable from common public entry points that application developers routinely use to validate, compare, and match versions:

- `Version.parse/1`
- `Version.parse!/1`
- `Version.match?/3`
- `Version.compare/2`
- `Version.parse_requirement/1`

The upstream patch is straightforward and important for defenders reviewing compensating controls. Commit `c64417d72fd5c7d09e963ca3ac5fa2b140978d9e` introduces `@max_numeric_component_digits 14`, rejects numeric components longer than 14 digits, updates module documentation to state the limit, and adds regression tests covering oversized numeric version components.

4.3 Exposure profile and affected-product context

Exposure pattern	Why it matters	Priority
Internet-facing Elixir / Phoenix services that parse client-supplied version fields, constraints, or package metadata	A single crafted request or payload may trigger expensive integer conversion and process instability	Immediate
Internal APIs, package-processing services, CI jobs, or dependency tooling that compare untrusted version strings	Even non-internet-facing workflows can be abused by semi-trusted upstream data sources or partner submissions	High

Azure Linux 3.0 systems relying on `elixir` `1.16.1-1`	Microsoft explicitly tracks this package as known affected and fixes it in `1.16.1-2`	High
Debian estates on vulnerable package lines (`bullseye`, `bookworm`, `trixie`, `forky`)	Distro lag means patch validation cannot rely on upstream version semantics alone	High
Applications that never parse untrusted version strings or that already enforce strict input-length limits	Reachable attack surface is materially reduced even before full patch rollout	Reduced

4.4 Defensive significance and ATT&CK framing

MITRE ATT&CK mapping is conditional because CVE-2026-49762 describes a parser weakness rather than a named intrusion chain. The mappings below are intended to frame defensive implications and telemetry, not to imply confirmed adversary tradecraft already observed in the wild.

ATT&CK ID	Technique	Rationale
T1190	Exploit Public-Facing Application	Applies when an exposed service accepts attacker-controlled version strings through HTTP or API parameters and passes them into the vulnerable parser.
T1499	Endpoint Denial of Service	Best-fit impact framing because the practical outcome is CPU and memory exhaustion or process crash rather than confidentiality or integrity loss.

4.5 Indicator assessment

Indicator category	Assessment
Stable network IOC set	None identified from reviewed public reporting.
Stable file hash IOC set	None identified; this is a vulnerable-code condition, not a malware family or fixed exploit kit.
Best operational indicators	Oversized semver-like request parameters, repeated parser failures referencing `Version` or `SystemLimitError`, and package inventories showing Elixir versions below the fixed boundary.

5. Threat Context

CVE-2026-49762 should be treated as an application-layer denial-of-service risk that becomes materially important wherever untrusted version strings cross a trust boundary. Many organizations use semantic-version parsing in places that appear operationally harmless at first glance: API compatibility checks, manifest validation, package-index normalization, plugin or extension metadata, CI/CD rule evaluation, software bill of materials processing, and dependency-policy enforcement. Those workflows can all become trigger points if they pass attacker-influenced strings into Elixir's vulnerable `Version` functions.

The public CVSS vector rates the issue as local because the vulnerable primitive lives inside a local parser, but that should not be misread as "not remotely relevant." In practice, remotely supplied HTTP parameters, JSON bodies, uploaded manifests, package records, or webhook content often flow into local parsing functions. Defenders should therefore prioritize reachable business logic, not just the raw CVSS access-vector label.

Reviewed public sources did not identify CISA KEV inclusion, named threat-actor reporting, or widespread observed exploitation. That lowers confidence in broad opportunistic exploitation at the time of review, but it does not remove operational risk. A single moderately sized all-digit string can be enough to pin a scheduler or crash a process, making the

flaw attractive for nuisance disruption, targeted service degradation, or low-noise availability attacks against exposed Elixir-backed workflows.

5.1 Representative abuse scenarios

Scenario	Description	Defensive meaning
API compatibility endpoint abuse	An attacker sends oversized `version`, `constraint`, or package-field values to a public endpoint that validates version syntax in Elixir	Review request parsing paths and rate-limit oversized semver-like input
Package or dependency metadata poisoning	Upstream package metadata, manifest fields, or repository records contain maliciously long numeric components	Treat external metadata feeds as hostile input, not trusted text
CI/CD or automation disruption	A malicious pull request, artifact, or build manifest triggers version parsing inside internal automation	Internal workflows can be exploitable even without internet exposure
Downstream distro lag	Operators assume OS-packaged Elixir is safe without verifying vendor backports or package versions	Patch validation must include vendor package state, not just upstream release notes

6. Detection and Hunting

Detection for CVE-2026-49762 should focus on exposure discovery, oversized semver-like input, parser-failure telemetry, and version inventory. No stable attacker infrastructure or malware-family IOC set was identified during this review. This is therefore an exposure-management and malformed-input hunting problem rather than a classic domain/IP/hash blocking problem.

6.1 Detection coverage summary

Signal	Telemetry source	Analytic goal	Coverage caveat
Elixir or distro package versions below the fixed boundary	Package inventory, SBOM, container manifests, config-management data	Identify systems requiring patch validation	Vendor backports and repackaged builds can obscure true fix state
Oversized semver-like values in request parameters or manifest fields	Reverse-proxy logs, application telemetry, API gateway logs, WAF logs	Surface likely exploit attempts and dangerous user-input paths	Field names vary widely across environments
`SystemLimitError`, `Version.parse`, or `Version.Parser` failures	Application logs, crash logs, APM traces, stderr capture	Detect blocked or successful parser exhaustion attempts	Benign parser bugs can produce similar strings
Repeated CPU spikes or process restarts on Elixir services receiving malformed input	APM, orchestrator events, service-health telemetry	Correlate availability issues with suspicious version parsing	Requires service-level observability, not just endpoint EDR
Azure Linux / Debian package lag	Fleet inventory, vulnerability management, package telemetry	Prioritize distro-packaged exposure	Distros may mark some releases as minor/no-DSA rather than urgent

6.2 Sigma - oversized version-parameter review in web or application logs

```
title: Suspicious Oversized Version Parameter for Elixir Version Parsing Review
```

```
id: 9bb8a67a-d2a8-4dd9-95f8-6be9c8b9e2d1
status: experimental
description: Hunts for very long numeric version-style parameters that may trigger CVE-2026-49762 in
exposed Elixir application logic. Field names should be adapted to the local web or application log
schema.
author: Lost Boys Cyber
date: 2026-06-15
references:
  - https://github.com/elixir-lang/elixir/security/advisories/GHSA-w2h8-8x3g-278p
  - https://cveawg.mitre.org/api/cve/CVE-2026-49762
logsource:
  category: webserver
  product: linux
detection:
  selection_param:
    cs-uri-query|contains:
      - 'version='
      - 'requirement='
      - 'constraint='
  selection_digits:
    cs-uri-query|re: '(?i)(^|[=&])(version|requirement|constraint)=[0-9]{15,}([+.-][0-9A-Za-z-]*)?'
  condition: selection_param and selection_digits
falsepositives:
  - Legitimate but unusually long numeric package metadata or test traffic
  - Benign parser fuzzing in development environments
level: medium
tags:
  - attack.t1190
  - attack.t1499
```

6.3 KQL - application-request hunt for oversized semver-like input

```
let lookback = 30d;
AppRequests
| where TimeGenerated > ago(lookback)
| extend DecodedUrl = url_decode(Url)
| where DecodedUrl matches regex @'(?i)(version|requirement|constraint)=[0-9]{15,}([+.-][0-9A-Za-
z-]*)?'
| project TimeGenerated, AppRoleName, Name, ResultCode, DurationMs, Success, ClientIP, Url=DecodedUrl
| order by TimeGenerated desc
```

6.4 KQL - parser failure and crash telemetry review

```
let lookback = 30d;
AppTraces
| where TimeGenerated > ago(lookback)
| where Message has_any ('SystemLimitError', 'Version.parse', 'Version.Parser', 'binary_to_integer')
| project TimeGenerated, AppRoleName, SeverityLevel, Message, OperationName, _ResourceId
| order by TimeGenerated desc
```

6.5 SPL - oversized version-field hunt and parser-error triage

```
(index=web OR index=app OR index=api)
(
  uri_query="*version=*" OR uri_query="*requirement=*" OR uri_query="*constraint=*"
  OR message="*SystemLimitError*" OR message="*Version.parse*" OR message="*Version.Parser*"
)
| eval candidate=coalesce(uri_query, request_query, url, message, _raw)
| regex candidate="(?i)(version|requirement|constraint)=[0-9]{15,}([+.-][0-9A-Za-z-]*)?"
| table _time host source sourcetype uri_path uri_query status user_agent message
| sort - _time
```

6.6 Hunt questions

Hypothesis	What to prove or disprove

Public or partner-facing services parse attacker-controlled version strings inside Elixir request handlers	Review route handlers, telemetry, and code paths that call `Version` helpers on external data
Existing logs already contain oversized semver-like values that were dismissed as malformed input noise	Search historical proxy, API gateway, and app logs for long all-digit version parameters
Fleet patching missed distro-packaged Elixir copies	Compare Azure Linux and Debian package inventories against the published fixed states
Availability incidents may already have been caused by parser exhaustion	Correlate restarts, CPU spikes, and crash traces with requests containing unusual version fields

No stable IOC set was identified from the reviewed sources. Hunting should focus on vulnerable code paths, package state, and anomalous input rather than on infrastructure-based threat indicators.

7. Recommendations

Priority	Owner	Recommendation	Rationale
Critical	Platform / Application owners	Upgrade to Elixir `1.20.1` or vendor-confirmed backports carrying the same parser-limit fix	Removes the known vulnerable logic in the upstream `Version` parser
High	Linux / Infrastructure owners	Validate Azure Linux and Debian package versions explicitly rather than assuming upstream release parity	Downstream packaging lag changes actual exposure state
High	Application developers	Reject or length-limit attacker-influenced version strings before calling `Version.parse*`, `Version.match?`, or `Version.parse_requirement/1`	Provides an immediate compensating control while patching rolls out
High	Security engineering	Add hunts and alerts for oversized semver-like request parameters plus `SystemLimitError` / parser failure telemetry	This is the most realistic detection surface for attempted abuse
High	DevOps / CI owners	Review internal automation, manifests, dependency pipelines, and package-processing jobs that parse external version strings	Semi-trusted upstream data can trigger the same flaw without internet exposure
Medium	Architecture / SRE	Rate-limit or isolate routes that evaluate attacker-controlled version metadata and ensure restart policies are tuned for repeated parser crashes	Reduces availability impact if exploit traffic reaches exposed paths
Medium	Detection engineering	Preserve example queries as sidecar artifacts and adapt field mappings to the local telemetry schema before operational	Generic hunting logic needs local normalization to be effective

		deployment	

Immediate next steps for most enterprises are straightforward:

- 1. Inventory Elixir runtimes and distro packages, especially Azure Linux and Debian estates, and confirm which systems remain below the fixed boundary.
- 2. Identify every route, job, manifest parser, or metadata-processing workflow that accepts untrusted version strings and calls Elixir `Version` functions.
- 3. Patch those systems or add strict input-length checks first, then validate with telemetry that oversized version parameters are rejected rather than parsed.
- 4. Review historical logs for `SystemLimitError`, `Version.Parser`, or unusually long semver-like request values to determine whether the issue has already been probed.

8. References

- [1] Microsoft Security Response Center: https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-49762
- [2] Microsoft June 2026 CVRF document: https://api.msrc.microsoft.com/cvrf/v3.0/cvrf/2026-Jun
- [3] GitHub security advisory `GHSA-w2h8-8x3g-278p`: https://github.com/elixir-lang/elixir/security/advisories/GHSA-w2h8-8x3g-278p
- [4] CVE JSON 5 / CNA record: https://cveawg.mitre.org/api/cve/CVE-2026-49762
- [5] OSV record `EEF-CVE-2026-49762`: https://osv.dev/vulnerability/EEF-CVE-2026-49762
- [6] NVD entry: https://nvd.nist.gov/vuln/detail/CVE-2026-49762
- [7] Elixir release `v1.20.1`: https://github.com/elixir-lang/elixir/releases/tag/v1.20.1
- [8] Upstream patch commit `c64417d72fd5c7d09e963ca3ac5fa2b140978d9e`: https://github.com/elixir-lang/elixir/commit/c64417d72fd5c7d09e963ca3ac5fa2b140978d9e
- [9] Debian Security Tracker: https://security-tracker.debian.org/tracker/CVE-2026-49762
- [10] Azure Linux upgrade guidance: https://learn.microsoft.com/en-us/azure/azure-linux/tutorial-azure-linux-upgrade